

EdgeGuard

AI-at-the-Edge Predictive Vehicle Health & Driver Copilot

Tata InnoVent 2026 · Category: AI at the Edge Solutions for Automotive

1. The Idea (30-second pitch)

EdgeGuard is a lightweight AI agent that runs at the vehicle's edge, processing sensor data locally in real time to predict failures before they happen, explain hazards in plain language, and act as a co-pilot that talks to the driver like a smart mechanic sitting in the passenger seat. Critical decisions are flagged instantly at the edge, with no cloud round-trip required; the system only syncs to a cloud/fleet dashboard for longer-term analytics.

2. Problem It Solves

- Vehicle faults (engine overheating, brake wear, tyre pressure loss) are usually caught only after they become dangerous or expensive.
- Drivers get cryptic dashboard warning lights with no context on severity or urgency.
- Fleet operators lack an easy way to aggregate vehicle health signals for predictive maintenance scheduling.

3. What It Actually Does

- **Synthetic Sensor Simulator:** Generates realistic real-time streams for engine temperature, RPM, brake pad wear %, tyre pressure, battery voltage, vibration, and speed. Includes injectable fault scenarios (overheating, critical brake wear, tyre pressure drop) that can be triggered live during the demo.
- **Edge Risk Engine:** A local threshold/rule-based scoring layer (the simulated 'edge AI') that continuously computes a live risk score from the sensor window, with zero cloud dependency for critical alerts.
- **Claude-Powered Copilot Layer:** Ingests the current sensor state every few seconds, detects anomalies, predicts time-to-failure, and generates a natural-language risk explanation. Acts as the 'cloud reasoning layer' that complements the fast local edge engine.
- **Driver Copilot Chat:** The driver (or judge) can type a question such as "Why is it warning me?" or "Is it safe to drive 50km?" and receive a real-time, contextual answer grounded in the current sensor readings.
- **Live Dashboard:** Gauges and charts for each sensor, a colour-coded risk meter, a scrolling predictive-maintenance alert feed, and the copilot chat, all on one screen. This visual is the centrepiece of the demo.
- **Fleet/ADAS Bonus View:** An additional card showing how the edge agent would sync anonymised alerts to a fleet-level dashboard for predictive fleet maintenance, extending the story to connected vehicles with minimal extra build time.

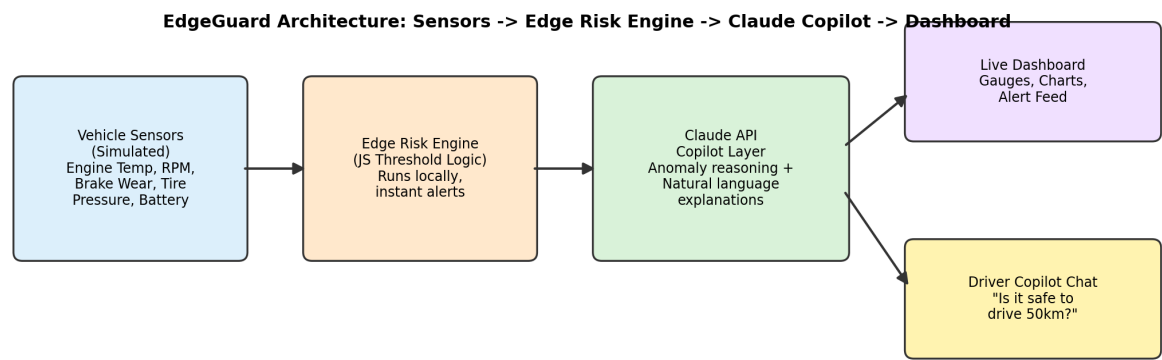
4. Why This Wins

- Directly hits 3 of the automotive sub-themes: ADAS/predictive maintenance, connected vehicles, and vehicle health monitoring, all in a single project.
- Visually impressive live demo (gauges + alerts + chat) that is fast to build with charts and the Claude API, with no real hardware required.
- Clear, architecturally credible 'edge' story: fast local reasoning on live data, cloud sync only for aggregation and deep reasoning.
- The live copilot chat is the 'wow' moment: judges get to talk to an AI that understands real-time vehicle context.

How to frame it for judges (say this during the pitch):

"EdgeGuard runs a lightweight risk-scoring model directly on the vehicle's edge unit, so critical alerts need no cloud round-trip. For deeper reasoning and natural-language driver interaction, it calls a cloud LLM layer, mirroring how real production systems split fast local inference from richer contextual reasoning. Our simulated sensor data follows real vehicle telemetry (OBD-II/CAN bus) patterns; in production, this edge agent would connect directly to the vehicle's sensor bus, e.g. via a Raspberry Pi or the vehicle ECU. Today's focus is proving the AI reasoning layer; hardware integration is the next phase."

5. Architecture Diagram



Flow: Simulated vehicle sensors feed the local Edge Risk Engine (instant threshold-based alerts) and, in parallel, the Claude API copilot layer (deeper anomaly reasoning and natural-language explanations). Both feed the live dashboard (gauges/charts/alerts) and the driver copilot chat interface.

6. Tech Stack

Layer	Technology	Purpose
Frontend / Dashboard	HTML + CSS + JavaScript (or React)	Gauges, cards, alert feed, overall UI
Charts	Chart.js	Live sensor graphs (temp, brake wear, tyre pressure, etc.)
Sensor Simulation	JavaScript setInterval	Generates fake real-time sensor data + fault triggers
Edge Risk Engine	JS if/else threshold logic	Plays the role of local 'edge AI' — instant risk scoring
Copilot Brain	Claude API (Anthropic /v1/messages)	Answers driver questions using live sensor context
Hosting	Netlify / Vercel (free tier) or local laptop	Quick, zero-cost deployment for the demo

7. Sensor Data Strategy

We are not connecting to a real vehicle; we use simulated/synthetic sensor data, which is standard for a 1-day PoC stage (real hardware integration typically comes 2-3 months later if shortlisted). Three options, from strongest to fastest:

- **Option 1 (Recommended):** Base the simulator on real public datasets, e.g. Kaggle/UCI 'Vehicle Sensor Data', 'Engine Health Dataset', or NASA Turbofan predictive-maintenance data, so temperature-rise curves and wear trends look realistic. Pitch line: “Our simulation is based on real-world automotive sensor patterns.”
- **Option 2:** Format the fake data as OBD-II (On-Board Diagnostics) style messages, the same format real cars actually transmit, to strengthen the 'production-ready format' story.
- **Option 3 (Fastest):** Pure rule-based random simulation with realistic value ranges. Fastest to build; adequate for a first working prototype.

8. Team Split (2 Members)

Member 1 — Frontend + Simulation	Member 2 — AI/Backend + Pitch
Build the dashboard: gauges/cards for temp, brake, tyre, battery	Integrate Claude API for the copilot chat, using live sensor state as context
Write the sensor simulator (random data generation + fault injection)	Define risk thresholds (when to warn, when to flag critical)
Add Chart.js live graphs	Write the Claude system prompt so it responds like a 'mechanic copilot'
Build the alert feed UI (warnings appear as risk rises)	Prepare the pitch deck / architecture diagram
Build the risk meter with colour transitions (green -> yellow -> red)	Rehearse the demo script and 2-3 canned questions to ask the copilot live

9. Timeline (1 Day, ~10-12 hours)

Time Block	Activity
First 3 hours	Both members together: build basic dashboard scaffold + fake sensor data foundation
Next 4-5 hours	Split work: Member 1 polishes UI/charts, Member 2 builds Claude integration + risk logic
Last 2-3 hours	Integrate both halves, test end-to-end, fix bugs, rehearse the pitch

10. Final Tips Before the Demo

- Screen-record a 60-90 second demo video as a backup in case live typing/API calls lag.
- Rehearse 2-3 canned questions to ask the copilot live so answers come fast and relevant.
- Keep the honesty framing ready: this is a software PoC with simulated telemetry; hardware integration is explicitly a next-phase step, which is normal and expected at this stage.